

As discussed previously, the concept that programs can be written using different algorithms to accomplish the same task, such as using iteration or recursion, is known as procedural abstraction. Similarly with data structures, the concept that when two different techniques can be used to represent a structure, such as using an array or references, this is known as data abstraction. In addition to lists, this concept of data abstraction can be extended to stacks and queues which were implemented using arrays in Chaps. 2 and 3, and will be implemented using references in this chapter. Since the basic concepts of stacks and queues were discussed previously, this chapter will concentrate on the implementation using references.

7.1 Stack

As should be recalled from Chap. 2, a stack is a LIFO (last-in, first-out) structure. Further, there were four methods developed for the class: `push`, `pop`, `empty`, and `full` plus a constructor. In this current section the constructor along with the `push` method will be developed leaving the `pop` and `empty` methods as exercises. Note that the `full` method is not included as will be discussed below. Assuming the existence of the `Node` class from Fig. 5.1, the basic skeleton for the class `StackRef` would look as follows:

```
public class StackRef {  
    private Node top;
```

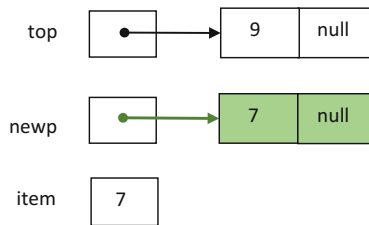
```
public StackRef() {  
    top = null;  
}  
}
```

This should look familiar in that it is similar to the basic structure of the `LinkedList` class except that instead of the private variable `head`, the variable `top` is used. Further, note that the constructor initializes `top` to `null` indicating an empty stack.

As in the last chapter, it might be helpful to start again with the generic case when pushing an item onto the stack. Assume that the stack already contains the number 9 and the number 7 will be pushed onto the stack. The first thing that needs to be done is to create a new node using the statement:

```
newp = new Node(item);
```

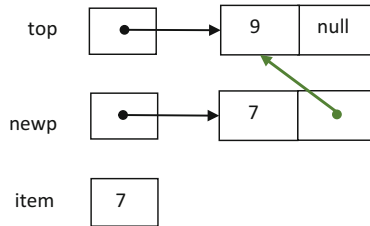
and as shown in the following drawing:



Then before altering the `top` reference, recall the heuristic from the previous chapter that it is usually better to create a new reference before altering any existing references. So the `next` part of the node referenced by `newp` needs to reference the node referenced by `top` using the following statement:

```
newp.setNext(top);
```

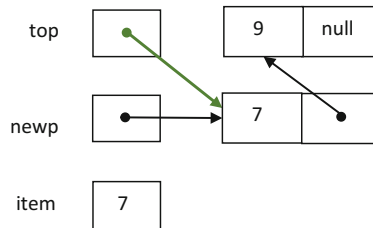
as shown in the following drawing:



Now that the new node is linked to the rest of the items in the stack, `top` can be changed to reference the new node as the top of the stack using the following:

```
top = newp;
```

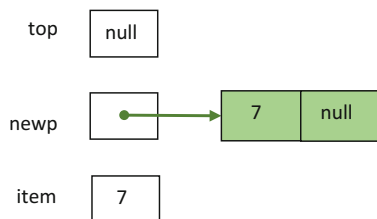
and as illustrated below:



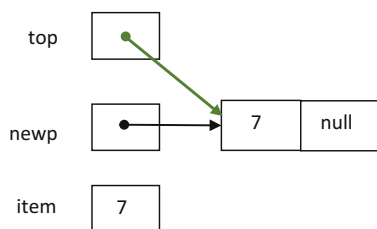
Putting the three lines together in the method `push` results in the following which can be added to `StackRef` class:

```
public void push(int item) {
    newp = new Node(item);
    newp.setNext(top);
    top = newp;
}
```

Will the above method work with an empty stack? Executing the first instruction in the method would result in the following:



Then executing the second instruction would copy the `null` reference in `top` to the `next` portion of the node referenced by `newp` which would look the same as the previous drawing because the constructor had already initialized the `next` portion to `null`. Lastly the third statement would cause `top` to refer to the same node as `newp` resulting in the following:



The result is correct which indicates the same code segment could be used for both an empty stack or a stack already containing items. Although the method could have probably been written for the empty stack first, this reinforces the technique used in the previous chapter.

Does one need to worry about whether the stack is full as is done with a stack implemented with arrays? Technically, it is possible that there are no more nodes available on the heap. However, since the stack here is only going to contain a small number of items, this should not be a concern. However, though the `full` method is not used for the `push` method, the `empty` method is most certainly needed when creating the `pop` method.

The partially complete class for `StackRef` is given in Fig. 7.1, Note that only the skeletons for the `pop` and `empty` methods are given, and they are left as exercises.

Again as mentioned earlier, the `StackRef` class in Fig. 7.1 assumes the existence of the `Node` class in Fig. 5.1.

```
public class StackRef {  
  
    private Node top;  
  
    public StackRef() {  
        top = null;  
    }  
  
    public void push(int item) {  
        Node newp = new Node(item);  
        newp.setNext(top);  
        top = newp;  
    }  
  
    public int pop() {  
        int item=-1;  
        // insert code  
    }  
  
    public boolean empty() {  
        // insert code  
    }  
}
```

Fig. 7.1 StackRef class

7.2 Stack Test Program: Reversing Integers

The beauty of data abstraction is that data structures can be implemented in classes using more than one way of storing the data and when using the class, one does not need to know the details. As a result, one should be able to use the program from Sect. 2.4 with very few changes. In fact, there are only three changes required to the main program. Two of these are that the references to the `StackArray` class need to be changed to `StackRef` as shown in the fifth and sixth lines below. Also, since there is theoretically no upper bound to the size of the stack, it is not necessary to check and the `&& !stack.full()` can be removed from the first while statement as shown below.

```
import java.util.*;  
class Reverse {  
    public static void main (String [] args){  
  
        int number;  
        StackRef stack;  
        stack = new StackRef();
```

```
Scanner scanner = new Scanner(System.in);

System.out.println();
System.out.print("Enter an integer or a -1 to stop: ");
number = scanner.nextInt();
while(number >= 0) {
    stack.push(number);
    System.out.print("Enter an integer or a -1 to stop: ");
    number = scanner.nextInt();
}

System.out.println();
System.out.print("The reverse integers are: ");
while(!stack.empty())
    System.out.print(stack.pop() + " ");
System.out.println();
System.out.println();
}
}
```

When running the program above with the new `StackRef` class, one will get similar results as before, without the limitation of the stack size as shown below:

```
Enter an integer or a -1 to stop: 1
Enter an integer or a -1 to stop: 2
Enter an integer or a -1 to stop: 3
Enter an integer or a -1 to stop: 4
Enter an integer or a -1 to stop: -1

The reverse integers are: 4 3 2 1
```

7.3 Queues

As with the `StackRef` class, the `QueueRef` class can implement the queue data structure using references. Recall that a queue is a FIFO (First In First Out) structure. Further, instead of a single variable `top`, two variables are needed, `front` and `rear`, which will be initialized to `null` in the constructor. In this

section, the enqueue method will be developed, and the dequeue and empty methods will be left as exercises. The basic skeleton class for the QueueRef class is as follows:

```
public class QueueRef {

    private Node front, rear;

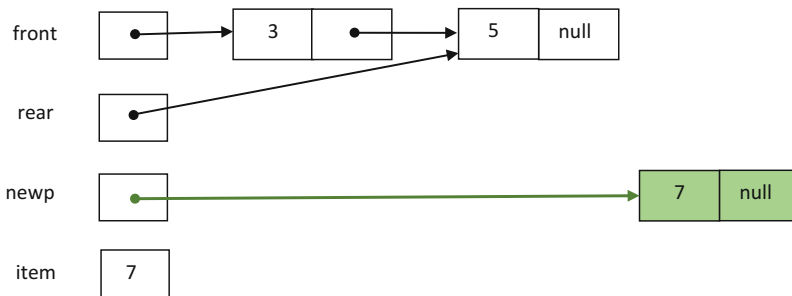
    public QueueRef() {
        front = null;
        rear = null;
    }

}
```

Starting again with a generic case, with 3 and 5 in the queue and ready to enqueue a 7, one would need to create a new node containing the item to be enqueued:

```
newp = new Node(item);
```

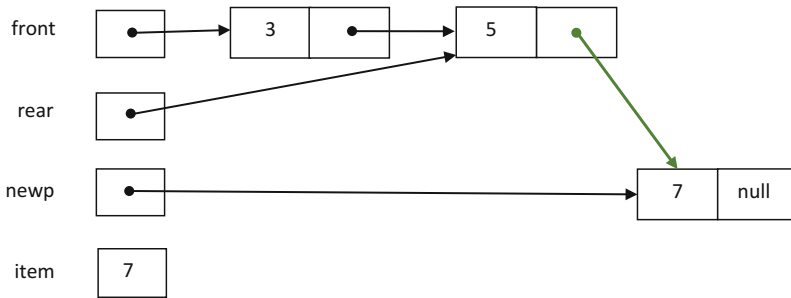
as illustrate below:



Note that the new node is drawn to the right to help facilitate the drawing in subsequent steps. Then the new item would be placed at the end of the queue by altering the next portion of the node referenced by `rear` to reference the same node referenced by `newp`:

```
rear.setNext(newp);
```

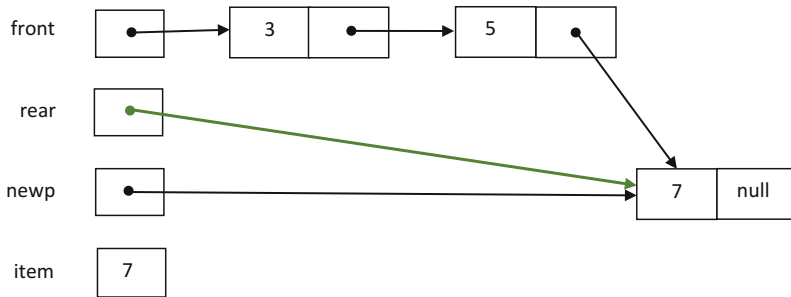
as shown below:



As before, note that the new reference is created before moving any other references. Now, the `rear` reference can be changed to reference the same node as `newp`:

```
rear = newp;
```

as illustrated below:

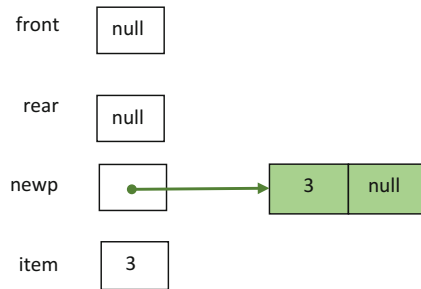


Putting these individual lines of code together creates the beginning of the `enqueue` method:

```
public void enqueue(int item) {
    newp = new Node(item);
    rear.setNext(newp);
    rear = newp;
}
```

As before, is a `full` method necessary? In this case there is no plan to enqueue very many items, so it is unlikely that the heap will run out of nodes. Similar to the stack discussed previously, does this code work with an empty queue? After

executing the first line of the method above, the following represents the state of execution:



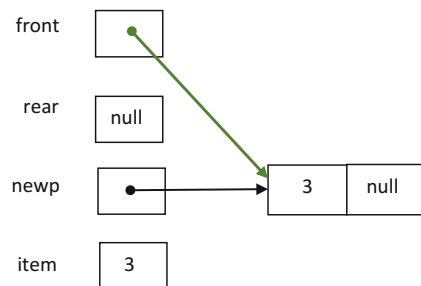
However, when trying to execute the second line of code, there would be an exception because `rear` is equal to `null`. As a result, an `if` statement needs to be inserted to ensure that `rear` is not equal to `null` before attempting to modify the node it is referencing.

```
if(rear != null)
    rear.setNext(newp);
```

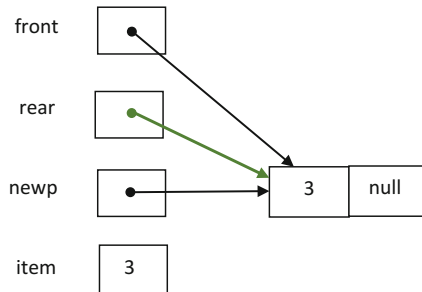
But if `rear` is equal to `null`, what should be modified? Instead of modifying the last node in the queue, `front` will need to reference the new node using the following code:

```
if(rear != null)
    rear.setNext(newp);
else
    front = newp;
```

and as illustrated below:



Then the last line in the method can be executed with `rear` also referencing the new node as shown below:



The result is that when inserting a node into an empty queue, it will be both the first and last node in the queue as can be seen with both `front` and `rear` referencing the same node. The new modified method for `insert` is given below:

```
public void enqueue(int item) {
    newp = new Node(item);
    if(rear != null)
        rear.setNext(newp);
    else
        front = newp;
    rear = newp;
}
```

The partially complete class for `QueueRef` is given in Fig. 7.2. Note that only the skeletons for the `dequeue` and `empty` methods are given and they are left as exercises.

As mentioned earlier with the `StackRef` class, the class in Fig. 7.2 assumes the existence of the `Node` class from Fig. 5.1.

```
public class QueueRef {  
  
    private Node front, rear;  
  
    public QueueRef() {  
        front = null;  
        rear = null;  
    }  
  
    public void enqueue(int item) {  
        Node newp = new Node(item);  
        if (rear != null)  
            rear.setNext(newp);  
        else  
            front = newp;  
        rear = newp;  
    }  
  
    public int dequeue() {  
        int item = -1;  
        // insert code  
    }  
  
    public boolean empty() {  
        // insert code  
    }  
  
}
```

Fig. 7.2 QueueRef class

7.4 Queue Test Program

As with stacks, queues can exhibit data abstraction, and the QueueArray class can be replaced with the QueueRef class. Using the program from Sect. 3.4 and assuming the existence of the Node class from Sect. 5.1, only two changes need to be made to the program. These are changing the class name QueueArray to QueueRef in the sixth and seventh lines as shown in the code below:

```
import java.util.*;  
public class TestQueueRef {  
    public static void main(String args[]) {  
        int item;  
        char selection;  
        QueueRef queue;  
        queue = new QueueRef();  
        Scanner scanner = new Scanner(System.in);
```

```

System.out.println();
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for Stop: ");
selection = scanner.next().charAt(0);
while(selection != 'S' && selection != 's') {
    if(selection == 'E' || selection == 'e') {
        System.out.print("Please enter an integer to be " +
"enqueued: ");
        item = scanner.nextInt();
        queue.enqueue(item);
    }
    else
        if(selection == 'D' || selection == 'd') {
            item = queue.dequeue();
            if(item != -1)
                System.out.println("The item dequeued is: " + item);
        }
    else
        System.out.println("The selection entered " + selection
+ " is not E, D, or S, please try again");
System.out.println();
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for Stop: ");
selection = scanner.next().charAt(0);
}
System.out.println();
System.out.println("End of Program");
System.out.println();
}
}

```

When running the program above with the new `QueueRef` class, one will get similar results as before, without the limitation of the queue size. Looking back at program in Sect. 3.4 which used the `QueueArray` class, note that the number 6 could not be enqueued without increasing the size of `qarray`; however using the `QueueRef` class there is no such restriction and note that number 6 is enqueued and dequeued below:

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
Queue is empty, item not removed

```

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 1

```

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 6

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 6

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
Queue is empty, item not removed

Please enter E for Enqueue, D for Dequeue, or S for Stop: s

End of Program

7.5 Comparison of Arrays Verses References

Which should be used for stacks and queues: arrays or references? The answer depends on the application. Given an equal number of items in a stack, which would take up more memory? The answer is the one with references because for each item in the stack, there needs to be an associated memory location referencing the next item in the stack. The same applies to queues. If this is the case, why would one want a stack or queue implemented using references? Recall from Chap. 2 that when using an array, it needs to be declared as a fixed size and there is always the possibility that the stack or queue could be declared too small and there could be an out of bounds exception. So this is an advantage to using a stack or queue implemented with references. However, one might also recall that code could be implemented that when a stack became full, a new larger array could be allocated and all the items in the old array could be copied into the new array and processing could continue. However, the problem with this solution is that the allocation and copying of the contents of an array would take time and slow down the algorithm.

The result is that if the maximum number of items that might be stored in a stack or queue is known in advance, then using an array based data structure would probably be the best choice due to the savings in memory. But if the number of items is not known in advance and could vary by quite a bit during each execution of the program, then a data structure implemented using references might be the better choice. So although data abstraction might hide the details of how a data structure is implemented, it is sometimes advantageous for a user of a class to understand how it is implemented to utilize the best one for the task at hand.

7.6 Complete Program: Undo Button

Consider the common feature “undo” which is a part of most applications software. Using the undo command, previous actions can be retrieved. It is very useful when an unwanted deletion or an unexpected modification has occurred while using software such as text editors. How can the undo feature be implemented in a program? To retrieve the actions already taken, they have to be stored in some way. Which data structure is appropriate to keep track of the history? When the undo is applied, the last action will be brought back. If the undo is applied again consecutively, the second last action needs to be recovered. So, as can be seen in the pattern, the answer to the question is a stack. In this section, buttons which change the background color of a small window and an undo button that retrieves the previous colors will be implemented with a stack using references as shown below.



7.6.1 Graphical User Interface

The programs developed so far use standard input and output. The interface with `System.in` and `System.out` is convenient and sufficient for basic programs. However, if a more user friendly interface is required a Graphical User Interface (GUI) can be employed. In Java, GUI is implemented using the standard classes defined in the `javax.swing` and `java.awt` packages. In order to build an effective GUI with objects from these predefined classes, a different style of program called event-driven programming is introduced. An event occurs when the user interacts with a GUI object. For example, when a user clicks a button or selects a menu option, an event occurs. In event-driven programs, objects that respond to these events are defined in an event-handling method. Next the basics of event-driven programming, which implements buttons and events when buttons are clicked, are discussed. For more information about the `javax.swing` and `java.awt` packages, refer to the References and Useful Websites section at the end of the text which has the link to the Java API specification document at the Oracle website.

7.6.2 User Defined Frame

To create an interactive user interface, a window is used where buttons are placed. A window is an instance of the `JFrame` class and the `JFrame` class contains minimum functionalities to support features found in any frames. Therefore, in order to design a window which performs customized meaningful tasks, a subclass of the `JFrame` class is declared where methods and data members are added to implement the necessary functionalities.

For example, using the `setTitle` method, the frame's title can be set. The `setSize` method can be used to set the frame's size, the `setLocation` method to position the frame's top left corner to coordinate (x, y), and `setBounds` to specify the size and the location of the frame. To terminate the program when the

frame is closed, the `setDefaultCloseOperation` method with the class constant `EXIT_ON_CLOSE` as an argument can be used. In this section only the `setSize` and the `setDefaultCloseOperation` methods are used as shown below:

```
setSize(300, 210);
setDefaultCloseOperation(EXIT_ON_CLOSE);
```

In the above, the size of the frame is 300 pixels wide and 210 pixels high and the program terminates when the Close box located at the top right corner of the window is clicked. These properties are set inside the default constructor of the user defined class which will be called `UndoButton`. A stack with objects and references, which is one of the main topics in this chapter, is used to implement an undo button. A main method will be included in this class so that a separate main class will not have to be defined. Here is the `UndoButton` class:

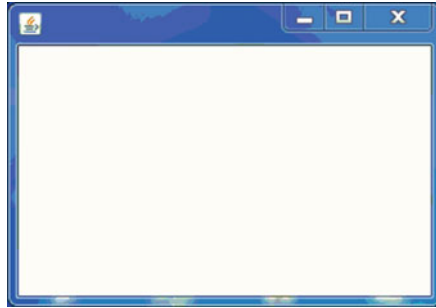
```
import javax.swing.*;

class UndoButton extends JFrame {
    public UndoButton() {
        // set the frame size
        setSize(300, 210);

        // exit upon closing the frame
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }
}
```

In the main method, an instance of `UndoButton` class is declared and created. An argument `true` is sent to the `setVisible` method to have the frame appear on the screen. When the above program is executed, the following window appears at the top left corner of the screen. If the different location is desired, the `setLocation` method can be used to specify the placement of the window.



7.6.3 Placing Buttons

There are two key aspects involved in GUI programming. One is the placement of GUI objects on the content pane of a frame, and the other is the handling of events generated by these GUI objects. In this section, a customized user interface is defined to show how four buttons labeled Blue, Red, Green and Undo are placed on the content pane of the frame. Then in the next section, how the button events are processed to change the background color and to undo the background color will be explained.

To display buttons and change the background color, the content pane of a frame needs to be accessed. A content pane is the area of the frame where the content such as buttons and images are displayed. It excludes the title, a menu bar, and the border. The content pane of a frame is accessed by calling the frame's `getContentPane()` method as shown below:

```
Container contentPane = getContentPane();
```

To deal with buttons, instances of the `javax.swing.JButton` class are created. The button with a label "Blue" can be declared and created in the following manner:

```
JButton blue;  
blue = new JButton("Blue");
```

The text passed as an argument to the constructor is the label of a button. In the complete program four buttons are placed by specifying their position and size explicitly on the content pane. This approach is called absolute positioning. Since a

layout manager for a container, which is an object that controls the placement of the GUI objects, is not used here, an argument `null` is sent to the method `setLayout`.

```
contentPane.setLayout(null);
```

The position and the size of the buttons are set using the `setBounds` method. For example, the Blue button is located 100 pixels to the right and 10 pixels down from the top left corner and the size is 100 pixels wide by 30 pixels high as follows:

```
blue.setBounds(100, 10, 100, 30);
```

To be able to see a button in the window, it has to be added to the content pane as in:

```
getContentPane().add(blue);
```

Here is the program to display four buttons in a frame.

```
import javax.swing.*;
import java.awt.*;

class UndoButton extends JFrame {
    private JButton blue, red, green, undo;

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }

    public UndoButton() {
        Container contentPane = getContentPane();

        // set the frame size
        setSize(300, 210);

        // set the content pane properties
        contentPane.setLayout(null);

        // create and place four buttons
        // on the frame's content pane
        blue = new JButton("Blue");
        blue.setBounds(100, 10, 100, 30);
        getContentPane().add(blue);
```

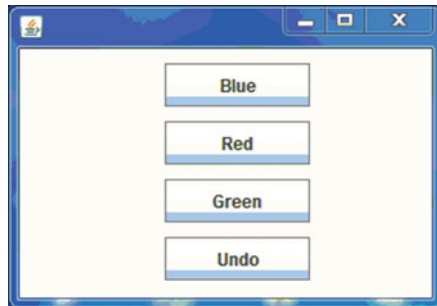
```
red = new JButton("Red");
red.setBounds(100, 50, 100, 30);
getContentPane().add(red);

green = new JButton("Green");
green.setBounds(100, 90, 100, 30);
getContentPane().add(green);

undo = new JButton("Undo");
undo.setBounds(100, 130, 100, 30);
getContentPane().add(undo);

// exit upon closing the frame
setDefaultCloseOperation(EXIT_ON_CLOSE);
}
}
```

When the program is executed, four buttons appear on the screen as shown below. These buttons can be clicked, but nothing happens because the code to handle the button clicks is not yet added to the class.



By default, the background is gray in color. To change it to white, for example, `setBackground` method can be used as in:

```
contentPane.setBackground(Color.white);
```

7.6.4 Handling Button Events

An action such as clicking a button is called an event and the mechanism to process the event is called event handling. Event handling is implemented by two types of objects: event source objects and event listener objects.

A GUI object, such as a button, where the event occurs is called an event source. When the user clicks a button, the corresponding `JButton` object will generate an event. When an event is generated, the system notifies the relevant event listener object that responds to the events by executing a method.

Among the many different types of events when a button is clicked, an event source will generate an action event. For the event to be processed, an event listener needs to be associated to the event sources. Otherwise, generated events are simply ignored. That is what happened in Sect. 7.6.3, where clicking buttons did not do anything. There are different kinds of listeners for each type of event: an action listener for action events, a window listener for window events, a mouse listener for mouse events, and so on.

To process an action event, an action listener class needs to include an `actionPerformed` method. To ensure this, the class must implement the `ActionListener` interface. Therefore, the `UndoButton` class is declared in the following way, a subclass of `JFrame` that implements the `ActionListener` interface.

```
class UndoButton extends JFrame implements ActionListener {  
    // implementation of UndoButton class  
}
```

Depending on which button is clicked, the background of the window will change. This is done inside the `actionPerformed` method. An argument to the `actionPerformed` method is an `ActionEvent` object that represents an action event, where the `ActionEvent` class contains methods to access the properties of a generated event. The general idea of the method is as follows:

```
public void actionPerformed(ActionEvent event) {                               // Line 1  
    String buttonText, color;                                                  // Line 2  
    JButton clickedButton = (JButton)event.getSource();                      // Line 3  
    buttonText = clickedButton.getText();                                       // Line 4  
  
    if(buttonText.equals("Undo"))                                              // Line 5  
        // process undo button  
    else                                                                        // Line 6  
        // process blue, red, and green buttons  
}
```

The text of the clicked button can be retrieved as in Line 3 and 4. The text of the blue button is the string “Blue”, the text of the red button is the string “Red”, and so on. Notice the typecasting of an object returned by the `getSource` method to `JButton`. This is because an object returned by the `getSource` method can be an instance of any class.

7.6.5 Generic Linked Stack

In order to keep the history of the command, the background color will be stored in the stack. Since the color of the background is of type `String`, the `StackRef` class which can be used only for integers is modified to accommodate items of any type. The `StackRefGeneric<T>` class that implements the `StackGeneric<T>` interface discussed in Sect. 2.7 is defined as shown in Fig. 7.3.

Remember that the class which implements an interface must have definitions for any methods listed in the interface. Although the list of objects linked together will never become full, and there is no use for the `full` method since it is included in the `StackGeneric<T>` interface for an array implementation of a stack, the code for the `full` method is added here to simply return the `false`. As before, the `pop` and `empty` methods are left as an exercise for the reader at the end of the chapter.

```
public class StackRefGeneric<T> implements StackGeneric<T> {
    private NodeGeneric<T> top;

    public StackRefGeneric(){
        top = null;
    }

    public boolean empty() {
        // insert code
    }

    public boolean full() {
        return false;
    }

    public T pop() {
        T item = null;
        // insert code
    }

    public void push(T item) {
        NodeGeneric<T> newp = new NodeGeneric<T>(item);
        newp.setNext(top);
        top = newp;
    }
}
```

Fig. 7.3 `StackRefGeneric < T >` class

7.6.6 The UndoButton Class

Because the `UndoButton` class is a subclass of `JFrame` that implements the `ActionListener` interface, a frame object is also an action event listener. Therefore, in the constructor the `addActionListener` method of the button is called by passing `this` as an argument. An example for the blue button is shown below:

```
blue.addActionListener(this);
```

If a user clicks either the Blue, Red, or Green button, the background color changes and the color is pushed onto the stack. This is done in the `setBackground` method. Inside the `actionPerformed` method, when a button for the different color is clicked the `setBackground` method is called. If the undo button is clicked, first it checks whether the stack is empty. The stack could be empty if the action was the very first one or if the undo operation emptied the stack previously. Then the program outputs a message. If the stack is not empty, it pops the current background color from the stack. After the top item is removed, the stack could be empty, since the user could have clicked one of the background color buttons only once or the undo button has been clicked and there was only the current color on the stack. If the stack is empty, the program will output a message. If it is not, the undo operation is performed to display the last background color in the window. The portion of the program which deals with the undo button is shown below:

```
if (buttonText.equals("Undo"))
    if (undoStack.empty())
        JOptionPane.showMessageDialog(null, "cannot undo");
    else {
        undoStack.pop();
        if (undoStack.empty())
            JOptionPane.showMessageDialog(null, "cannot undo");
        else {
            color = undoStack.pop();
            setBackground(color);
        }
    }
}
```

The complete listing of the `UndoButton` class which is a subclass of `JFrame` that implements the `ActionListener` interface discussed in Sect. 7.6.4 uses the `NodeGeneric<T>` class in Fig. 5.6 and the `StackRefGeneric<T>` class from Sect. 7.6.5 is shown in Fig. 7.4. An instance of the `StackRefGeneric<String>` class is created and used to store the color of the background every time the user changes it.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class UndoButton extends JFrame implements ActionListener {
    private JButton blue, red, green, undo;
    private StackRefGeneric<String> undoStack
        = new StackRefGeneric<String>();

    public static void main(String[] args) {
        UndoButton frame = new UndoButton();
        frame.setVisible(true);
    }

    public UndoButton() {
        Container contentPane = getContentPane();

        // set the frame size
        setSize(300, 200);

        // set the content pane properties
        contentPane.setLayout(null);
        contentPane.setBackground(Color.white);

        // create and place four buttons on the frame's content pane
        blue = new JButton("Blue");
        blue.setBounds(100, 10, 100, 30);
        getContentPane().add(blue);

        red = new JButton("Red");
        red.setBounds(100, 50, 100, 30);
        getContentPane().add(red);

        green = new JButton("Green");
        green.setBounds(100, 90, 100, 30);
        getContentPane().add(green);

        undo = new JButton("Undo");
        undo.setBounds(100, 130, 100, 30);
        getContentPane().add(undo);

        // associate an action listener to the four buttons
        blue.addActionListener(this);
        red.addActionListener(this);
        green.addActionListener(this);
        undo.addActionListener(this);

        // exit upon closing the frame
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent event) {
```

Fig. 7.4 UndoButton class

```

String buttonText, color;
JButton clickedButton = (JButton) event.getSource();
buttonText = clickedButton.getText();

if (buttonText.equals("Undo"))
    if (undoStack.empty())
        JOptionPane.showMessageDialog(null, "cannot undo");
    else {
        undoStack.pop();
        if (undoStack.empty())
            JOptionPane.showMessageDialog(null, "cannot undo");
        else {
            color = undoStack.pop();
            setBackgroundColor(color);
        }
    }
else
    setBackgroundColor(buttonText);
}

private void setBackgroundColor(String color) {
    if (color.equals("Blue")) {
        getContentPane().setBackground(Color.blue);
        undoStack.push("Blue");
    }
    else
        if (color.equals("Red")) {
            getContentPane().setBackground(Color.red);
            undoStack.push("Red");
        }
        else
            if (color.equals("Green")) {
                getContentPane().setBackground(Color.green);
                undoStack.push("Green");
            }
    }
}

```

Fig. 7.4 (continued)

7.7 Summary

- Hiding how a data structure is implemented, whether using arrays or references, is known as data abstraction.
- Using references to implement stacks and queues can take up more memory, but the data structure can grow or shrink as needed.
- Although `full` is not necessarily needed with stacks and queues implemented using references, an `empty` method is definitely needed.
- In order to build an effective graphical user interface (GUI), event-driven programming is used.

- In order to process the action events generated by a button, an action listener needs to be associated to the button.

7.8 Exercises (Items Marked with an * Have Solutions in Appendix B)

1. Assuming the stack `intStack` of type `StackRef` in Fig. 7.1 was initially empty, draw a stack after the following operations are completed:

```
int x, y, z;
intStack.push(41);
intStack.push(6);
x = intStack.pop();
y = intStack.pop();
intStack.push(78);
intStack.push(5);
intStack.push(79);
z = intStack.pop();
intStack.push(5);
```

- *2. Assuming the stack `characterStack` of type `StackRefGeneric<Character>` which is an object of the generic class `StackRefGeneric<T>` in Fig. 7.3 is initially empty, draw a stack after the following operations are completed:

```
Character x, y, z;
characterStack.push('U');
characterStack.push('V');
x = characterStack.pop();
characterStack.push('W');
y = characterStack.pop();
characterStack.push('P');
characterStack.push('Q');
z = characterStack.pop();
characterStack.push('C');
```

3. Assuming the queue `intQueue` of type `QueueRef` in Fig. 7.2 is initially empty, draw a queue after the following operations are completed:

```

int x, y, z;
intQueue.enqueue(41);
intQueue.enqueue(6);
x = intQueue.dequeue();
y = intQueue.dequeue();
intQueue.enqueue(78);
intQueue.enqueue(5);
intQueue.enqueue(79);
z = intQueue.dequeue();
intQueue.enqueue(5);

```

4. Implement a `QueueRefGeneric<T>` class including all the data members and methods from the `QueueRef` class in Fig. 7.2.
- *5. Assuming the queue `characterQueue` of type `QueueRefGeneric<Character>` which is an object of the generic class `QueueRefGeneric<T>` created in problem 4 is initially empty, draw a queue after the following operations are completed:

```

Character x, y, z;
characterQueue.enqueue('U');
characterQueue.enqueue('V');
x = characterQueue.dequeue();
characterQueue.enqueue('W');
y = characterQueue.dequeue();
characterQueue.enqueue('P');
characterQueue.enqueue('Q');
z = characterQueue.dequeue();
characterQueue.enqueue('C');

```

6. Write a method `pop` discussed in Sect. 7.1 that can be added to the `StackRef` class in Fig. 7.1.
7. Write a method `empty` that can be added to the `StackRef` class in Fig. 7.1 which returns `true` if the list is empty.
8. Write a method `pop` discussed in Sect. 7.1 that can be added to the `StackRefGeneric<T>` class in Fig. 7.3.
9. Write a method `empty` that can be added to the `StackRefGeneric<T>` class in Fig. 7.3 which returns `true` if the list is empty.
10. Write a method `dequeue` discussed in Sect. 7.3 that can be added to the `QueueRef` class in Fig. 7.2.
11. Write a method `empty` that can be added to the `QueueRef` class in Fig. 7.2 which returns `true` if the list is empty.